

EASY MEMORY

NAND, NOR, XNOR columns are the opposites of the AND, OR, XOR columns. Learn AND, OR, XOR. Then just flip the column.

5.3 Every gate in detail

● LEVEL 1 • BEGINNER

AND gate

$$Y = A \cdot B$$



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Analogy: Two switches in a row (series) with a bulb. The bulb glows only when **both** switches are ON.

How it works: Output is 1 only when **all** inputs are 1.

Exam points: Any input 0 gives output 0. $A \cdot 1 = A$. $A \cdot 0 = 0$. $A \cdot A' = 0$.

Common mistake: Writing $1 \cdot 1 = 2$. In Boolean algebra $1 \cdot 1 = 1$.

Quick trick: One 0 kills AND. Look for a 0 first.

OR gate

$$Y = A + B$$



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Analogy: Two switches side by side (parallel). The bulb glows if **either** switch is ON.

How it works: Output is 0 only when **all** inputs are 0. Otherwise 1.

Exam points: Any input 1 gives output 1. $A + 0 = A$. $A + 1 = 1$. $A + A' = 1$.

Common mistake: Writing $1 + 1 = 10$ or 2. In Boolean algebra $1 + 1 = 1$.

Quick trick: One 1 wins in OR. Look for a 1 first.

NOT gate (inverter)

$$Y = A'$$



A	Y
0	1
1	0

Analogy: A light that is ON when the switch is OFF. It always says the opposite.

How it works: Flips the input: 0 becomes 1, 1 becomes 0. It is the only gate with **one input**.

Exam points: The bubble means "invert". Two NOT gates in a row cancel: $(A')' = A$.

Common mistake: Saying NOT has two inputs, or leaving out the bubble in a diagram.

Quick trick: NOT = flip.

NAND gate

$$Y = (A \cdot B)'$$



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Analogy: An alarm that is always ON, and it switches OFF only when **both** buttons are pressed.

How it works: First AND, then NOT. Output is 0 only when **all** inputs are 1.

Exam points: Universal gate. $(AB)' = A' + B'$. NAND with inputs joined together = NOT.

Common mistake: Writing $(AB)' = A'B'$. Correct: $(AB)' = A' + B'$ (De Morgan).

Quick trick: NAND = flip the AND column. Only 1,1 gives 0.

NOR gate

$$Y = (A + B)'$$



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Analogy: A room light that turns ON only when **both** switches are OFF.

How it works: First OR, then NOT. Output is 1 only when **all** inputs are 0.

Exam points: Universal gate. $(A + B)' = A' \cdot B'$. NOR with inputs joined together = NOT.

Common mistake: Writing $(A + B)' = A' + B'$. Correct: $A'B'$.

Quick trick: NOR = flip the OR column. Only 0,0 gives 1.

XOR gate (exclusive OR)

$$Y = A \oplus B = A'B + AB'$$



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Analogy: A staircase light with a switch at the bottom and another at the top. Flipping *either* switch changes the light.

How it works: Output is 1 when the two inputs are **different**. Same inputs give 0.

Exam points: $A \oplus 0 = A$. $A \oplus 1 = A'$. $A \oplus A = 0$. $A \oplus A' = 1$. **n inputs: output is 1 when the number of 1s is odd** (odd parity). Used in adders, parity and comparators.

Common mistake: Mixing XOR with OR. They are the same except for input 1,1 (OR gives 1, XOR gives 0).

Quick trick: Count the 1s. **Odd number of 1s → output 1**. Also: XOR = OR without the (1,1) row.

XNOR gate (exclusive NOR)

$$Y = (A \oplus B)' = AB + A'B'$$



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Analogy: Two students answering a question. The light is green if their answers are the **same**.

How it works: Output is 1 when the two inputs are the **same**. Different inputs give 0.

Exam points: 2-input XNOR = equality checker. XNOR = NOT of XOR. **For n inputs (as NOT of XOR): output 1 when the number of 1s is even.**

Common mistake: Thinking a 3-input XNOR means "all inputs equal". It does not; it is the complement of the 3-input XOR.

Quick trick: XNOR = same → 1. Flip the XOR column.

Buffer

$$Y = A$$



A	Y
0	0
1	1

Analogy: A pipe that does not change the water but pushes it with more pressure.

How it works: Output = input. It changes nothing in logic, but it makes the signal **stronger** so that it can drive more gates.

Exam points: Used to increase fan-out, to add a small delay, and (as a **tri-state buffer**) to connect or disconnect a wire: output can be 0, 1 or Z (high impedance = “disconnected”).

Common mistake: Thinking a buffer is useless because it does not change the value.

Quick trick: Triangle, no bubble = buffer.

XOR	XNOR
1 when inputs are different	1 when inputs are the same
$A'B + AB'$	$AB + A'B'$
Odd number of 1s gives 1	Even number of 1s gives 1
Used in adders (sum bit), parity generators	Used in equality comparators

5.4 Gate properties (good for MCQs)

● LEVEL 2 · EXAM

Gate	Commutative (A,B = B,A)	Associative (grouping)	Universal?
AND, OR	Yes	Yes	No
XOR, XNOR	Yes	Yes	No
NAND	Yes	No	Yes
NOR	Yes	No	Yes

GATE POINT

NAND and NOR are **not associative**: $(A \text{ NAND } B) \text{ NAND } C \neq A \text{ NAND } (B \text{ NAND } C)$. So a 3-input NAND cannot be made by just chaining two 2-input NANDs.

QUICK TRICK

Bigger gates from 2-input gates (AND, OR, XOR): an n -input gate needs $(n - 1)$ two-input gates. 8-input AND = 7 gates. The levels (depth) needed = $\lceil \log_2 n \rceil$. 8 inputs $\rightarrow$ 3 levels.

QUICK TRICK

Gate with inputs joined: $\text{AND}(A,A) = A$, $\text{OR}(A,A) = A$ (buffer). $\text{NAND}(A,A) = A'$, $\text{NOR}(A,A) = A'$ (NOT). $\text{XOR}(A,A) = 0$. $\text{XNOR}(A,A) = 1$.

QUICK TRICK

XOR as a controlled inverter: $A \oplus 1$ flips A . $A \oplus 0$ keeps A . This is used in the adder-subtractor circuit later.

GATE POINT

Number of different Boolean functions of n variables = 2^{2^n} . $n = 2 \rightarrow 2^4 = 16$ functions (AND, OR, NAND, NOR, XOR, XNOR and 10 more). $n = 3 \rightarrow 2^8 = 256$.

5.5 Universal gates: NAND and NOR

● LEVEL 2 • EXAM

What does universal mean? With only NAND gates (or only NOR gates), you can build NOT, AND, OR and every other gate. **Why do we care?** In real chips it is cheaper to make one kind of gate many times.

USING NAND ONLY	GATES NEEDED
NOT A' = NAND(A, A)	1
AND AB = NAND(NAND(A,B) , NAND(A,B)) (NAND, then NOT)	2
OR $A+B$ = NAND(NAND(A,A) , NAND(B,B)) (NOT both, NAND)	3
XOR Y = N1 = NAND(A,B); N2 = NAND(A,N1); N3 = NAND(B,N1); Y = NAND(N2, N3)	4

USING NOR ONLY	GATES NEEDED
NOT A' = NOR(A, A)	1
OR $A+B$ = NOR(NOR(A,B) , NOR(A,B)) (NOR, then NOT)	2
AND AB = NOR(NOR(A,A) , NOR(B,B)) (NOT both, NOR)	3

QUICK TRICK

Easy way to remember: NAND builds AND in **2** gates (NAND + NOT) and OR in **3** gates. NOR builds OR in **2** gates (NOR + NOT) and AND in **3** gates. The gate that “matches” (AND for NAND, OR for NOR) is the easy one.

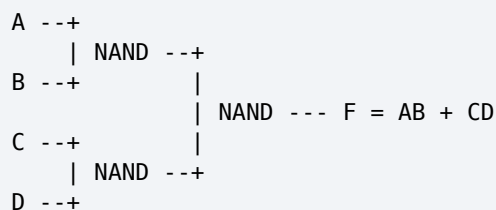
EASY MEMORY

Counting trick: NOT = 1 gate. The “matching” gate (AND for NAND, OR for NOR) = 2 gates. The other gate = 3 gates. XOR from NAND = 4 gates.

Any function with NAND only — the double-bubble trick

A circuit in **SOP form** (AND gates feeding an OR gate) can be rebuilt with **only NAND gates**: replace **every** gate by a NAND. The two bubbles cancel (De Morgan), so the function stays the same. For **POS form** (OR gates feeding an AND) use **only NOR gates**.

$F = AB + CD$ with NAND only (3 gates):



Why does it work? (Level 3)

NAND-NAND output = $((AB)' \cdot (CD)')'$

Use De Morgan on the outside bar: $(AB)'' + (CD)'' = AB + CD$. ✓

Example: $F = (A + B)(C + D)$ with NOR only

POS form → use NOR-NOR. Gate 1 = NOR(A,B), gate 2 = NOR(C,D), output = NOR(gate 1, gate 2).

Check: $((A+B)' + (C+D)')' = (A+B)(C+D)$. ✓ Gates needed: **3**.

QUICK TRICK

SOP → NAND-NAND. POS → NOR-NOR. The number of gates = number of product (or sum) terms + 1 (plus NOT gates for complemented inputs).

5.6 Reading a circuit

● LEVEL 3 • GATE

Method: write the output of each gate, from the inputs toward the output, one gate at a time. Then simplify with Boolean laws from Chapter 4.

```

A  --+
   |  NAND  --+
B  --+       |  NOR  --- Y
C  -----+
  
```

Find Y

Gate 1 (NAND): $X = (AB)'$

Gate 2 (NOR): $Y = (X + C)' = ((AB)' + C)'$

Use De Morgan: $Y = (AB)'' \cdot C' = AB \cdot C' = \mathbf{ABC'}$

So $Y = 1$ only when $A = 1$, $B = 1$ and $C = 0$.

QUICK TRICK

Bubble-push trick (De Morgan in pictures): a NAND gate is the same as an **OR gate with bubbles on the inputs**. A NOR gate is the same as an **AND gate with bubbles on the inputs**. Use this to simplify circuits quickly: $(AB)' = A' + B'$ and $(A + B)' = A'B'$.

Quick-Solve Tricks — Summary

Gate	Trick to get the output
AND	One 0 kills it $\rightarrow 0$. Only all 1s gives 1.
OR	One 1 wins $\rightarrow 1$. Only all 0s gives 0.
NOT	Flip.
NAND	Flip AND. Only all 1s gives 0.
NOR	Flip OR. Only all 0s gives 1.
XOR	Count the 1s. Odd $\rightarrow 1$. (Two inputs: different $\rightarrow 1$.)
XNOR	Flip XOR. Even number of 1s $\rightarrow 1$. (Two inputs: same $\rightarrow 1$.)
Bubble	A bubble = NOT.
Universal building	NOT = 1 gate, matching gate = 2, other gate = 3, XOR = 4 (NAND).
n-input gate from 2-input gates	$(n - 1)$ gates.
SOP / POS with one gate type	SOP $\rightarrow$ NAND-NAND. POS $\rightarrow$ NOR-NOR.

GATE QUICK CHECK — Chapter 5

AND: $Y = AB$ OR: $Y = A + B$ NOT: $Y = A'$
 NAND: $Y = (AB)'$ NOR: $Y = (A + B)'$
 XOR: $Y = A'B + AB'$ XNOR: $Y = AB + A'B'$
 $X \text{ xor } 0 = X$ $X \text{ xor } 1 = X'$ $X \text{ xor } X = 0$ $X \text{ xor } X' = 1$
 Functions of n variables = $2^{(2^n)}$ n-input gate from 2-input gates = $n - 1$

Traps: $(AB)' \neq A'B'$ · NAND and NOR are not associative · XNOR with more than 2 inputs · positive-logic AND = negative-logic OR · XOR and OR differ only at (1,1).

Practice questions

● LEVEL 1 · BEGINNER

Q1. What is the output of an OR gate with inputs 0 and 1?

Q2. How many inputs does a NOT gate have?

Q3. Name the two universal gates.

● LEVEL 2 · EXAM

Q4. A 4-input XOR gate has inputs 1, 1, 0, 1. Find the output.

Q5. How many 2-input AND gates are needed to make an 8-input AND gate?

Q6. How many 2-input NAND gates are needed to build a 2-input OR gate?

● LEVEL 3 · GATE

Q7. A NAND gate takes A and B. Its output and C go to a NOR gate. Find Y and simplify.

Q8. Simplify $A \oplus B \oplus AB$.

Q9. How many 2-input NAND gates are needed (minimum) to build a 2-input XOR gate?

Q10. Using only 2-input NAND gates, how many are needed (minimum) to build a 3-input NAND gate?

Q11. A positive-logic NOR gate is used in negative logic. Which gate is it now?

Q12. Let the 3-input XNOR be defined as $(A \oplus B \oplus C)'$. What is the output for inputs 1, 1, 1?

Answers

Q	Solution
1	One 1 wins in OR → 1 .
2	One input.
3	NAND and NOR .
4	Count the 1s: three 1s (odd) → XOR = 1 .
5	$n - 1 = 7$ gates (4 + 2 + 1 in three levels).
6	OR = NAND(NAND(A,A), NAND(B,B)) → 3 gates.
7	$X = (AB)'$. $Y = (X + C)' = ((AB)' + C)' = AB \cdot C' \rightarrow Y = \mathbf{ABC'}$.
8	Truth table: (A,B) = (0,0) → 0; (0,1) → $1 \oplus 0 = 1$; (1,0) → 1; (1,1) → $0 \oplus 1 = 1$. That is the OR function → A + B .
9	$N1 = (AB)'$, $N2 = (A \cdot N1)'$, $N3 = (B \cdot N1)'$, $Y = (N2 \cdot N3)' \rightarrow$ 4 gates.
10	3-input NAND = $(ABC)'$. Step 1: NAND(A,B) and NAND it with itself to get AB (2 gates). Step 2: NAND(AB, C) = $(ABC)'$ (1 gate). Total 3 gates.
11	Duality: positive NOR = negative NAND .
12	$A \oplus B \oplus C$ with three 1s (odd) = 1. XNOR = $1' =$ 0 .

Next: Chapter 6 — Boolean Functions: truth table to expression, SOP, POS, minterms, maxterms, Σ and Π notation, don't-cares.